STUDENT INFORMATION SYSTEM (SIS)

Ain Shams University - Faculty of Engineering

Department of Electronics and electronic communications Engineering

Introduction to computer programming CSE141

2026

Submitted by:

Nour Ashraf Mohamed	25P0182
Mohamed Eid Ramadan.	25P0029
Joyse Fayez	25P0026
Youssef Wael	25P0209

Submitted to:

Prof. Mahmoud Khalil
Eng. Meryam Hani

Student Information System

Project Report

This report explains the design, implementation, features, validation rules, file handling, and main code logic of the C++ Student Information System project.

Prepared By

Nour Ashraf Mohamed	25P0182
Youssef Weal Ibrahim	25P0209
Mohamed Eid Ramadan	25P0029
Joyse Fayez	25P0026

Contents

1 Introduction

The Student Information System (SIS) is a console-based C++ application designed to manage student records, course records, course registration, grades, GPA calculation, and transcript generation. The system represents a simplified version of a university information system and focuses on structured data storage, input validation, menu-driven interaction, and persistent saving using CSV files.

The project applies several core C++ concepts, including structures, arrays, functions, file handling, formatted output, condition checking, and modular programming. Each module is separated by responsibility, making the program easier to understand, test, and improve.

2 Project Objectives

The main objectives of the project are:

- Manage student information such as name, national ID, phone number, program, level, GPA, and registered courses.
- Manage course information such as course code, title, and credit hours.
- Register courses for students while preventing duplicate or invalid registrations.
- Enter midterm and final grades, then calculate total marks, letter grades, and grade points.
- Calculate GPA based on grade points and credit hours.
- Generate a transcript in a formatted table.
- Save all important data into CSV files and reload it when the program starts again.

3 Required Libraries

3.1 Included Libraries

Library	Purpose in the Project
`<iostream>`	Handles input and output using `cin` and `cout`.
`<string>`	Stores and manipulates text data such as names, IDs, phone numbers, and course codes.
`<fstream>`	Reads from and writes to CSV files.
`<sstream>`	Splits lines read from CSV files into separate fields.

`<iomanip>`	Formats output tables using `setw`, `left`, and decimal precision.
`<ctime>`	Gets the current year for student age checking and automatic ID generation.
`<cstring>`	Copies C-style strings, especially for the date of birth character array.
`<limits>`	Helps clear invalid input from the input buffer.

3.2 Main Include Section

```
#define _CRT_SECURE_NO_WARNINGS
#include <limits>
#include <iostream>
#include <cstring>
#include <ctime>
#include <string>
#include <fstream>
#include <iomanip>
#include <sstream>
using namespace std;
```

4 System Overview

When the program starts, it loads saved data from three CSV files: `students.csv`, `courses.csv`, and `grades.csv`. After loading the data, the main menu appears and allows the user to choose one of the main modules.

Main modules: Student Management, Course Management, Grades Management, and Exit.

4.1 Main Menu Flow

1. The user runs the program.
2. The program loads saved students, courses, and grades.
3. The main menu is displayed.
4. The user selects the required module.
5. The selected module opens a submenu.
6. Data is saved automatically after important operations.

5 System Features

5.1 Student Management Features

The Student Management module includes:

- Add a new student.
- List all students and sort them by ID, name, or GPA.
- Search students by student ID, name, or national ID.
- Update student phone number, program, or academic level.
- Delete a student after confirmation.

5.2 Course Management Features

The Course Management module includes:

- Add a new course.
- View all courses in a formatted table.
- Update course title or credit hours.
- Delete a course after confirmation.
- Register a course to a student.

5.3 Grades Management Features

The Grades Management module includes:

- Enter student grades.
- View all grades for a selected student.
- Calculate GPA.
- Generate a student transcript.

6 Data Structures

The project uses structures to group related data together. This makes the data easier to store, access, and pass between functions.

6.1 Student Structure

The `Student` structure stores personal, academic, and registration information for each student.

```
struct Student
{
    string name;
    string nationalID;
    char gender;
    char DOB[11];
    string phone;
    string program;
    int level;
    float GPA;
    string ID;
    string registeredCourses[10];
    int registeredCourseCount;
};
```

The system can store up to **1200 students** using the global array:

```
Student students[1200];
int studentCount = 0;
```

6.2 Course Structure

The `Course` structure stores the course code, course title, and number of credit hours.

```
struct Course
{
    string code;
    string title;
    int creditHours;
};
```

The system can store up to **200 courses**.

6.3 Grade Structure

The `Grade` structure stores the grade record for a student in a specific course.

```
struct Grade
{
    string studentID;
    string courseCode;
    float midterm;
    float finalExam;
    float total;
    char letterGrade;
```

```
    float points;
};
```

The system can store up to **5000 grade records**.

7 Menu Design

The program is menu-driven. Each menu asks the user to enter a number, then the program uses conditional statements or switch statements to call the correct function.

7.1 Main Menu Code Snippet

```
int MainMenu()
{
    int choice;

    cout << "1. Student Management\n";
    cout << "2. Courses Management\n";
    cout << "3. Grades Management\n";
    cout << "4. Exit\n";
    cout << "Enter your choice: ";
    cin >> choice;

    while (choice > 4 || choice < 1)
    {
        cout << "Invalid entry, enter choice between 1 and 4:
           ";
        cin >> choice;
    }

    if (choice == 1) studentManagement();
    else if (choice == 2) courseManagement();
    else if (choice == 3) gradesManagement();
    else if (choice == 4) return 0;

    return 0;
}
```

8 Input Validation

Input validation is one of the most important parts of the system. It protects the program from incorrect data and improves the reliability of saved records.

8.1 Name Validation

The student name must contain exactly two words, usually first name and last name. Only alphabetical characters and spaces are accepted.

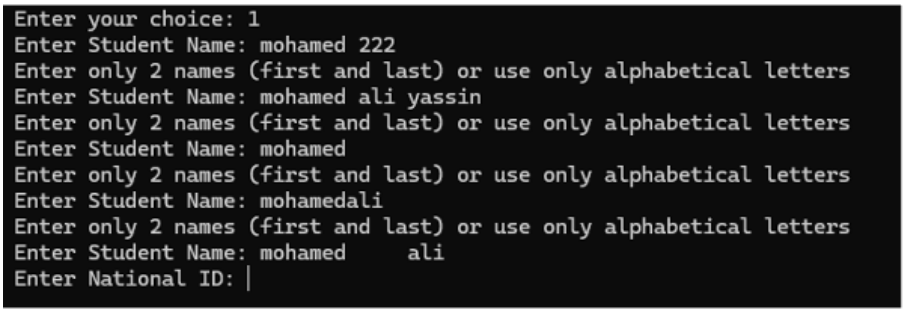

Figure 1: Invalid inputs for name validation.

8.2 National ID Validation

The national ID must be exactly 14 digits and must not start with zero. The system also checks that the national ID is unique and not already used by another student.

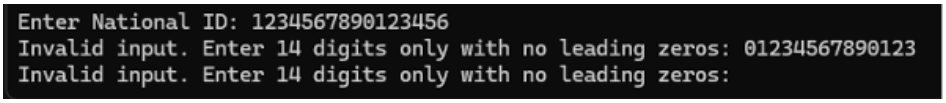

Figure 2: Invalid inputs for national id validation.

8.3 Gender Validation

The system accepts only `M`, `m`, `F`, or `f`.

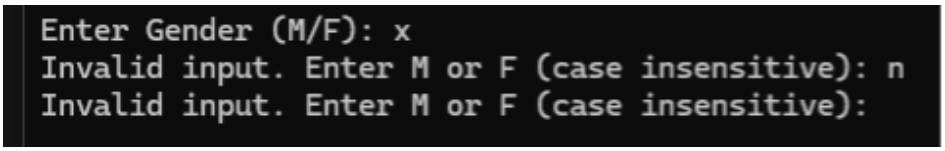

Figure 3: Invalid inputs for gender validation.

8.4 Date of Birth Validation

The date of birth must follow the format `DD/MM/YYYY`. The system checks the positions of the slashes, the ranges of the day and month, and the student's age. The student must be at least 17 years old.

```
1 bool isValid(char DOB[])
2 {
3     return (DOB[2] == '/' && DOB[5] == '/' &&
4         DOB[0] >= '0' && DOB[0] <= '3' &&
```

```
        DOB[1] >= '0' && DOB[1] <= '9' &&
        DOB[3] >= '0' && DOB[3] <= '1' &&
        DOB[4] >= '0' && DOB[4] <= '9' &&
        DOB[6] >= '1' && DOB[6] <= '2');
}
```

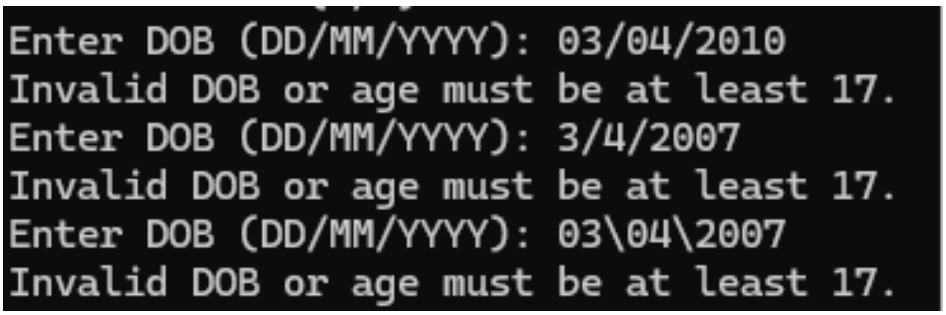

Figure 4: Invalid inputs for DOB validation.

8.5 Phone Number Validation

The phone number must be exactly 11 digits and must start with `01`.

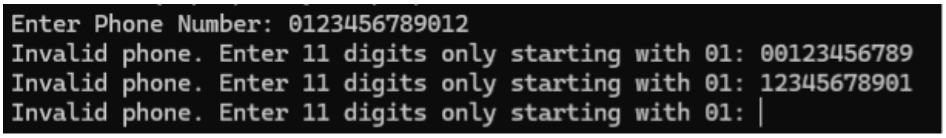

Figure 5: Invalid inputs for Phone validation.

8.6 Program Validation

The program must be one of the following values:

- CSE
- CCE
- MCT

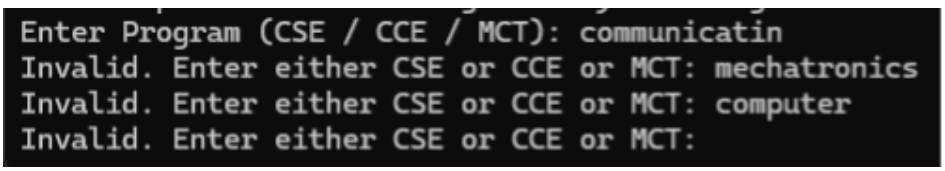

Figure 6: Invalid inputs for Program validation.

8.7 Academic Level Validation

The academic level must be between 1 and 4.

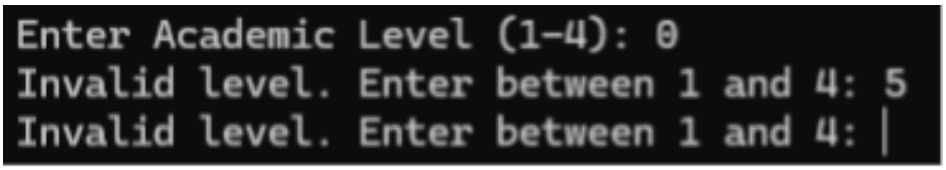

Figure 7: Invalid inputs for academic level validation.

8.8 Student ID Validation

The student ID must follow the format `XXPXXXX`, such as `26P0001`. The first two characters must be digits, the third character must be `P`, and the last four characters must be digits.

```
bool isValidStudentID(string id)
{
    if (id.length() != 7) return false;
    if (!(id[0] >= '0' && id[0] <= '9')) return false;
    if (!(id[1] >= '0' && id[1] <= '9')) return false;
    if (id[2] != 'P') return false;

    for (int i = 3; i < 7; i++)
        if (!(id[i] >= '0' && id[i] <= '9')) return false;

    return true;
}
```

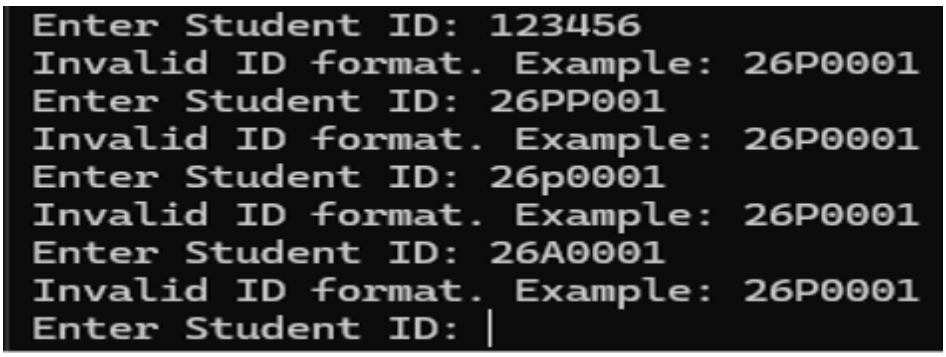

Figure 8: Invalid inputs for student ID validation.

8.9 Course Code Validation

The course code must contain exactly three uppercase letters followed by three digits. Example: `CSE141`.

```
bool isValidCourseCode(string code)
{
    if (code.length() != 6) return false;

    for (int i = 0; i < 3; i++)
        if (code[i] < 'A' || code[i] > 'Z') return false;

    for (int i = 3; i < 6; i++)
        if (code[i] < '0' || code[i] > '9') return false;

    return true;
}
```

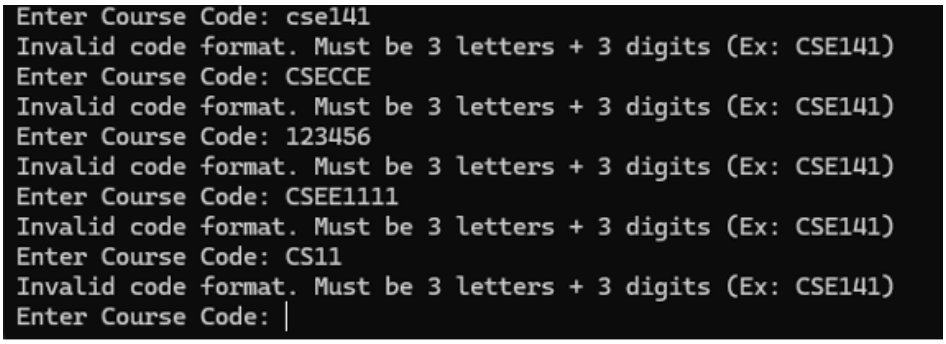

Figure 9: Invalid inputs for course code validation.

8.10 Grade Validation

Midterm grades must be between 0 and 40. Final exam grades must be between 0 and 60. After validating both values, the system calculates the total grade.

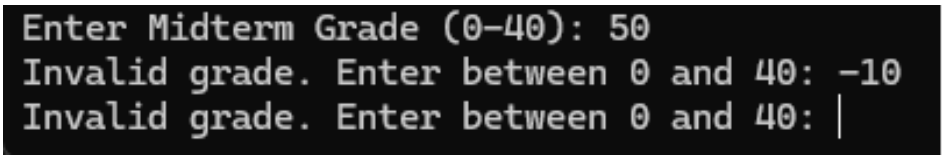

Figure 10: Invalid inputs for midterm exam grade validation.

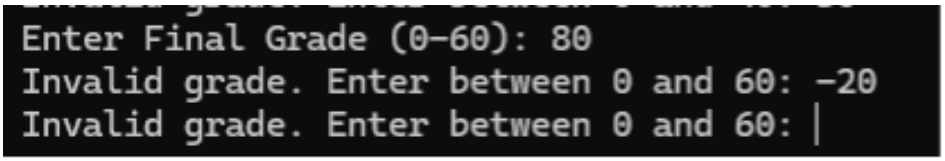

Figure 11: Invalid inputs for final exam grade validation.

9 Student Management Module

9.1 Adding a New Student

When adding a new student, the system asks the user to enter the student's name, national ID, gender, date of birth, phone number, program, and academic level. The GPA starts at 0.0 and the registered course count starts at 0.

After all inputs are valid, the system automatically generates a student ID. The ID is created using the current year and a serial number. For example, if the year is 2026 and the student is the first student, the generated ID is `26P0001`.

9.2 Listing Students

The system can display all students in a table. Before displaying, the user can choose one of three sorting methods:

- Sort by Student ID.
- Sort by Name from A to Z.
- Sort by GPA from highest to lowest.

9.3 Searching Students

The system supports three search methods:

- Search by Student ID.
- Search by Student Name.
- Search by National ID.

If a matching student is found, the system displays the student's full information inside a formatted box.

9.4 Updating Student Data

The user can update selected fields only:

- Phone number.
- Program.
- Academic level.

The system applies validation again before saving the updated data.

9.5 Deleting a Student

The system first searches for the student using the entered student ID. If the student exists, the system asks the user to confirm the deletion using `Y` or `N`. If confirmed, the student is removed from the array and the file is updated.

10 Course Management Module

10.1 Adding a Course

The user enters the course code, title, and credit hours. The course code must follow the correct format and must be unique. The title cannot be empty, and credit hours must be between 1 and 4.

10.2 Viewing Courses

All courses are displayed in a table containing course code, course title, and credit hours.

10.3 Updating a Course

The user enters the course code. If the course exists, the user can update either the course title or credit hours.

10.4 Deleting a Course

The system asks for the course code, searches for it, then asks for confirmation before deleting it.

10.5 Registering a Course to a Student

Before registering a course, the system checks four conditions:

1. The student must exist.
2. The course must exist.
3. The student must not already be registered in the course.
4. The student must not exceed 10 registered courses.

11 Grades Management Module

11.1 Entering Grades

When entering grades, the system asks for the student ID and course code. It checks that the student exists, the course exists, the student is registered in the course, and the grade record does not already exist.

The user then enters the midterm grade and final exam grade. The system validates both values, calculates the total, assigns the letter grade, and assigns grade points.

11.2 Letter Grade and Points

Total Mark	Letter Grade	Grade Points
90 to 100	A	4.0
80 to less than 90	B	3.0
70 to less than 80	C	2.0
60 to less than 70	D	1.0
Less than 60	F	0.0

```
char getLetterGrade(float total)
{
    if (total >= 90) return 'A';
    else if (total >= 80) return 'B';
    else if (total >= 70) return 'C';
    else if (total >= 60) return 'D';
    else return 'F';
}
```

11.3 Viewing Student Grades

The user enters a student ID, and the system displays all recorded grades for that student in a table. The table includes course code, midterm grade, final grade, total grade, and letter grade.

11.4 GPA Calculation

The GPA is calculated using grade points and credit hours. Each course contributes according to its credit hours.

$$GPA = \frac{\sum(GradePoints \times CreditHours)}{\sum CreditHours} \tag{1}$$

```
float GPA = totalPoints / totalCredits;
```

After calculation, the GPA is saved inside the student record and written to `students.csv`.

11.5 Generating a Transcript

The transcript displays the student's ID, name, program, level, course list, credit hours, letter grades, grade points, total credit hours, and cumulative GPA. It is printed in a formatted table to make it clear and readable.

12 File Handling

The project uses CSV files for permanent storage. This means the data does not disappear when the program closes.

12.1 Files Used

File Name	Data Stored
`students.csv`	Student information, GPA, and registered courses.
`courses.csv`	Course code, course title, and credit hours.
`grades.csv`	Student ID, course code, midterm, final, total, letter grade, and grade points.

12.2 Saving Student Data

```
void saveToFile(Student students[], int studentCount)
{
    ofstream file("students.csv");

    for (int i = 0; i < studentCount; i++)
    {
        file << students[i].name << ","
             << students[i].nationalID << ","
             << students[i].gender << ","
             << students[i].DOB << ","
             << students[i].phone << ","
             << students[i].program << ","
             << students[i].level << ","
             << students[i].GPA << ","
             << students[i].ID << "\n";
    }

    file.close();
}
```

12.3 Loading Data at Program Start

When the program starts, the `main()` function loads the saved students, courses, and grades before showing the main menu.

```
int main()
{
    loadFromFile(students, studentCount);
    loadCoursesFromFile();
    loadGradesFromFile();

    MainMenu();

    return 0;
}
```

13 Important Implementation Notes

13.1 Use of Arrays

The program uses fixed-size arrays for students, courses, and grades. This makes the implementation simple and suitable for an introductory C++ project. The maximum capacities are:

- 1200 students.
- 200 courses.
- 5000 grade records.
- 10 registered courses per student.

13.2 Use of Functions

The project uses functions to divide the program into smaller parts. This improves readability and makes the project easier to debug. Examples include `studentManagement()`, `courseManagement()`, `gradesManagement()`, `saveToFile()`, `loadFromFile()`, and `calculateGPA()`.

13.3 Use of Formatted Output

The project uses `setw`, `left`, and `setprecision` from `<iomanip>` to display data in clean tables.

13.4 Visual Studio Note

The line `#define _CRT_SECURE_NO_WARNINGS` is used mainly with Microsoft Visual Studio to avoid warnings related to some C/C++ functions. It is not a separate library and does not affect the main logic of the program.

14 Program Workflow

1. Start the program.
2. Load saved CSV files.
3. Display the main menu.
4. Choose Student Management, Course Management, or Grades Management.
5. Perform the required operation.
6. Validate the entered data.
7. Save updates to the correct CSV file.
8. Return to the main menu or exit.

15 Limitations and Possible Improvements

The current system works correctly for the required console-based project, but it can be improved in future versions.

15.1 Current Limitations

- The system uses fixed-size arrays instead of dynamic containers.
- The interface is console-based, not graphical.
- CSV files are simple and easy to use, but they are not as powerful as a database.
- Some deleted or updated records may require extra handling if connected data exists in other files.

15.2 Future Improvements

- Use `vector` instead of fixed arrays.
- Add login accounts for admins and users.
- Add a graphical user interface.

- Use a database instead of CSV files.
- Export transcripts as PDF files.
- Add more detailed error handling.

16 Project Links

The project files and source code are available through the following links:

16.1 OneDrive Link

The OneDrive folder contains the project files, report files, and related project materials.

https://engasuedu-my.sharepoint.com/:f:/g/personal/25p0209_eng_asu_edu_eg/IgD_cAXRs63AQpJ2NvWDhhEHAbnI36F7O2fCJ8vg4IH-88Y?e=cYH9aq

16.2 GitHub Repository

The GitHub repository contains the C++ source code of the Student Information System project.

https://github.com/YoussefWael92/cpp-project/tree/main/code

17 Conclusion

The Student Information System project demonstrates how C++ programming concepts can be used to build a practical and organized university management system. The project manages students, courses, grades, GPA calculation, and transcripts using structures, arrays, functions, validation, and file handling.

The system successfully checks user input before saving data, prevents several common errors, and keeps records available after the program closes through CSV files. Overall, the project meets the main requirements of a simplified Student Information System and provides a strong base for future development.